
Expert Systems

Unit 7

Hidden Markov Model

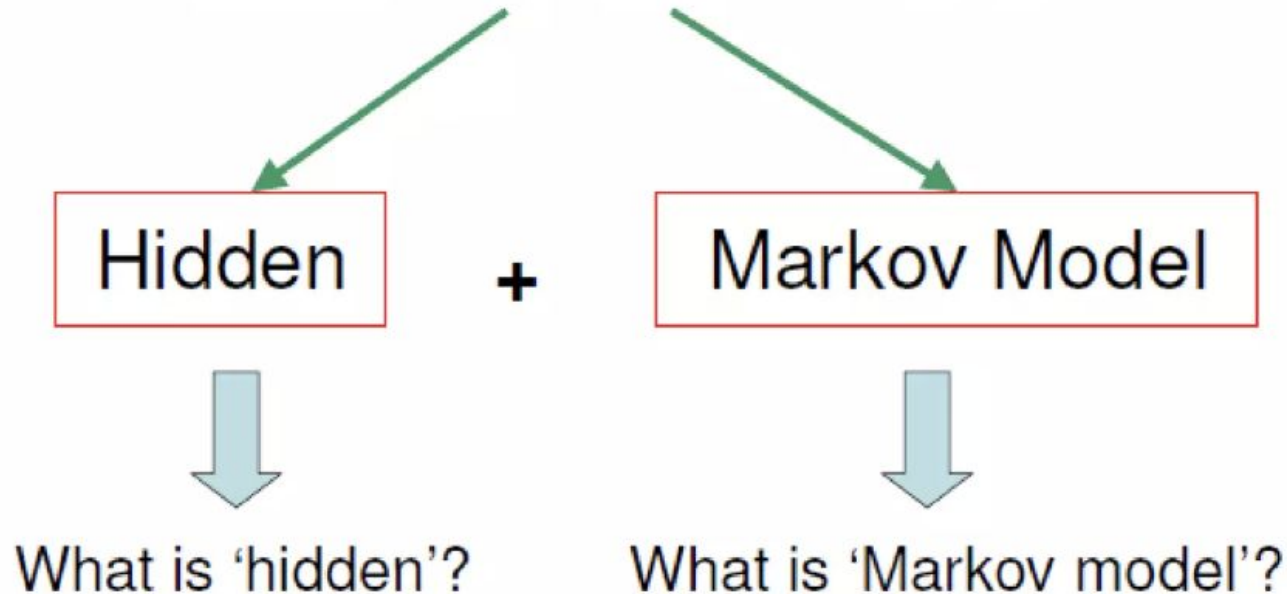
- A hidden Markov model (HMM) is a **statistical model**, in which the system being modeled is assumed to be a **Markov process** (Memoryless process: *its future and past are independent*) with ***hidden states***.

Hidden Markov Model

- Has a set of **states** each of which has limited number of **transitions** and **emissions**,
- Each transition between states has an assigned probability,
- Each model starts from start state and ends in end state,

Hidden Markov Model

Hidden Markov Model



Markov Model

- Talk about weather,
- Assume there are three types of weather:

– Sunny,



– Rainy,



– Foggy.



Markov Model

Weather prediction is about the what would be the weather tomorrow,

- Based on the observations on the past.



Markov Model

- Weather at day n is $q_n \in \{sunny, rainy, foggy\}$



- q_n depends on the known weathers of the past days ($q_{n-1}, q_{n-2}, \dots$)

Markov Model

- We want to find that:

$$P(q_n | q_{n-1}, q_{n-2}, \dots, q_1)$$

- means given the past weathers what is the probability of any possible weather of today.

Markov Model

- For example:

- if we knew the weather for last three days was:



- the probability that tomorrow would be  is:

$$P(q_4 = \text{cloud} \mid q_3 = \text{rain}, q_2 = \text{sunny}, q_1 = \text{sunny})$$

Markov Model and its assumption

- Therefore, make a simplifying assumption **Markov assumption**:

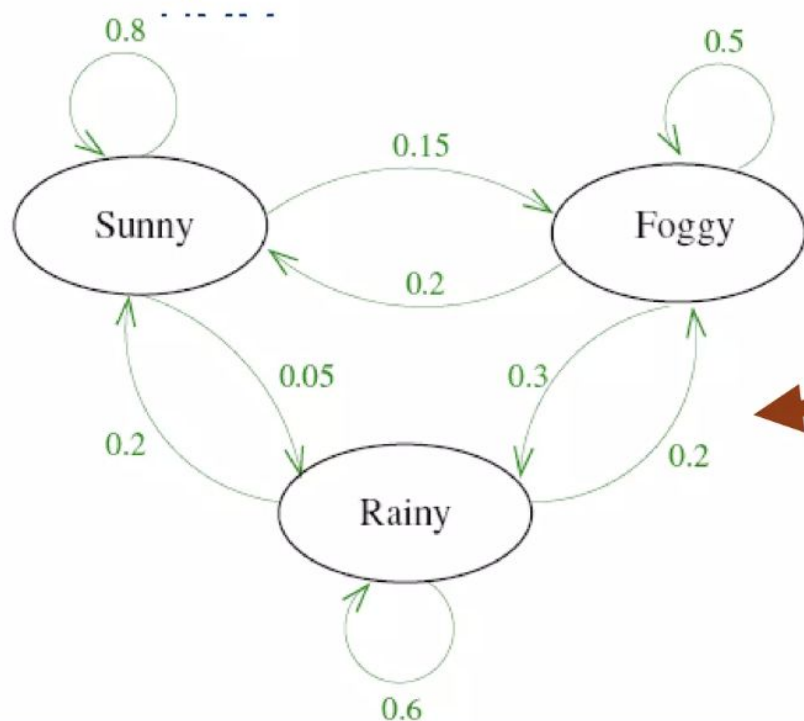
- For sequence: $\{q_1, q_2, \dots, q_n\}$

$$P(q_n | q_{n-1}, q_{n-2}, \dots, q_1) = P(q_n | q_{n-1})$$

- **the weather of tomorrow only depends on today**
(first order Markov model)

Markov Model and its assumption

Examples:



Today's weather	Tomorrow's weather		
			
	0.8	0.05	0.15
	0.2	0.6	0.2
	0.2	0.3	0.5



Markov Model and its assumption

Examples:

- If the weather yesterday was rainy and today is foggy what is the probability that tomorrow it will be sunny?



Markov Model and its assumption

– Examples:

- If the weather yesterday was rainy and today is foggy what is the probability that tomorrow it will be sunny?



$$P(q_3 = \text{☀} | q_2 = \text{☁}, q_1 = \text{☁}) = P(q_3 = \text{☀} | q_2 = \text{☁}) = 0.2.$$

Markov assumption

Hidden Markov Model

– What is HMM:

- Suppose that you are locked in a room for several days,
- you try to predict the weather outside,
- The only piece of evidence you have is whether the person who comes into the room bringing your daily meal is carrying an umbrella or not.

Hidden Markov Model

- What is HMM (cont.):
 - assume probabilities as seen in the table:

Weather	Probability of umbrella
Sunny	0.1
Rainy	0.8
Foggy	0.3

Probability $P(x_i|q_i)$ of carrying an umbrella ($x_i = \text{true}$)
based on the weather q_i on some day i

Hidden Markov Model

- What is HMM (cont.):
 - Finding the probability of a certain weather

$$q_n \in \{sunny, rainy, foggy\}$$



- is based on the observations $\mathbf{x}_i$:

Hidden Markov Model

- What is HMM (cont.):
 - Using Bayes rule:

$$P(q_i|x_i) = \frac{P(x_i|q_i)P(q_i)}{P(x_i)}$$

- For n days:

$$P(q_1, \dots, q_n|x_1, \dots, x_n) = \frac{P(x_1, \dots, x_n|q_1, \dots, q_n)P(q_1, \dots, q_n)}{P(x_1, \dots, x_n)}$$

Application of HMMs

Speech Recognition: HMMs are widely used in speech recognition systems to model the acoustic features of speech and recognize spoken words

Bioinformatics: Used for tasks like gene finding and protein structure prediction

Activity Recognition: Applied in video analysis to recognize human activities

Finance: Used for modeling and predicting financial time series

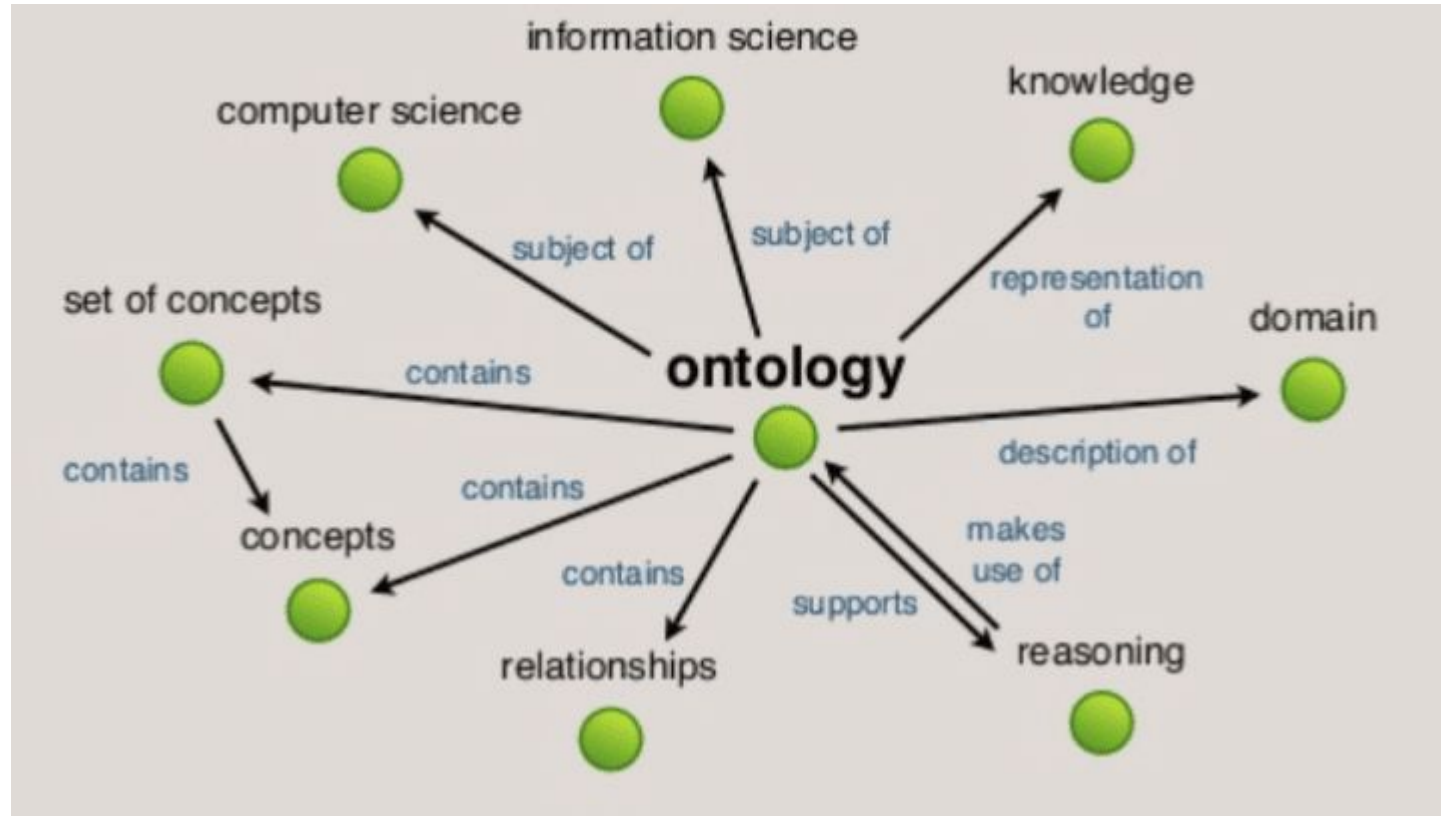
Ontology based knowledge representation

Ontology-based knowledge representation involves creating a structured framework to organize and share knowledge across different systems and applications.

It uses ontologies, which are formal descriptions of concepts and relationships within a domain, to provide a common understanding and vocabulary.

The term "ontological" refers to concepts and discussions related to ontology, which is the branch of philosophy that deals with the study of existence, being, and reality.

Ontology based knowledge representation



Expert Systems

Introduction

An Expert System is a collection of programmes or Computer Software that solves problems in the domain of interest.

It is called system because it consists of both problem solving component and a support component.

The process of building Expert System is called knowledge engineering and is done by knowledge Engineer

An Expert system is a set of program that manipulates encoded knowledge to solve problem in a specialized domain that normally requires human expertise.

A computer system that simulates the decision- making process of a human expert in a specific domain.

An expert system's knowledge is obtained from expert sources and coded in a form suitable for the system to use in its inference or reasoning processes. The expert knowledge must be obtained from specialists or other sources of expertise, such as texts, journals, articles and data bases.

An expert system is an “intelligent” program that solves problems in a narrow problem area by using high-quality, specific knowledge rather than an algorithm.

Expert systems provide the following important features:

Facility for non-expert personnel to solve problems that require some expertise

Speedy solution

Reliable solution

Cost reduction

Power to manage without human experts

Wider areas of knowledge

Use of expert system is especially recommended when

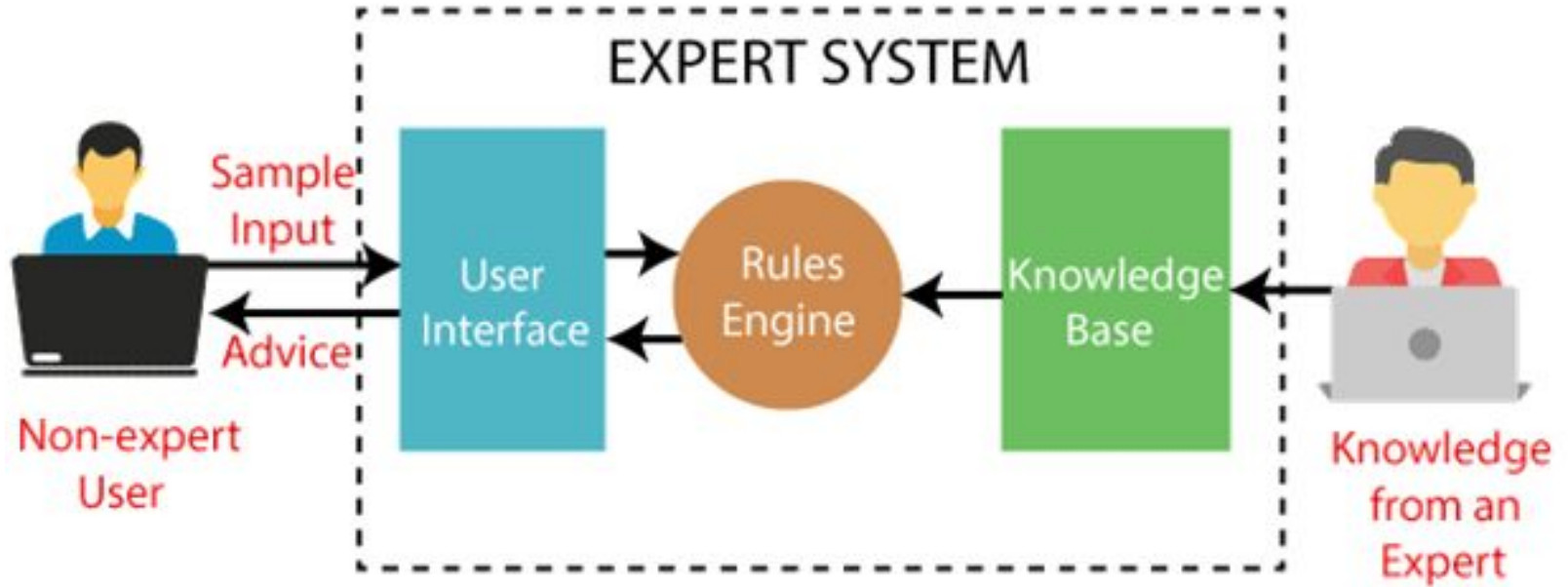
Human experts are difficult to find

Human experts are expensive

Knowledge improvement is important

The available information is poor, partial, incomplete

Problems are incompletely defined There is lack of knowledge among all those who need it The problem is rapidly changing legal rules and codes



1. Knowledge Base

The component of an expert system that contains the system's knowledge is called its knowledge base. This element of the system is so critical to the way most expert systems are constructed that they are also popularly known as *knowledge-based systems*

A knowledge base contains both declarative knowledge (facts about objects, events and situations) and procedural knowledge (information about courses of action). Depending on the form of knowledge representation chosen, the two

knowledge may be separate or integrated. Although many knowledge representation techniques have been used in expert systems, the most prevalent form of knowledge representation currently used in expert systems is the *rule-based production* system approach.

To improve the performance of an expert system, we should supply the system with some knowledge about the knowledge it possesses, or in other words, meta-knowledge.

2. Inference Engine

Simply having access to a great deal of knowledge does not make you an expert; you also must know how and when to apply the appropriate knowledge.

Similarly, just having a knowledge base does not make an expert system intelligent.

The system must have another component that directs the implementation of the knowledge. That element of the system is known variously as the *control structure*, the *rule interpreter*, or the *inference engine*.

The inference engine decides which heuristic search techniques are used to determine how the rules in the knowledge base are to be applied to the problem.

In effect, an inference engine “runs” an expert system, determining which rules are to be invoked, accessing the appropriate rules in the knowledge base, executing the rules , and determining when an acceptable solution has been found.

3. User Interface

The component of an expert system that communicates with the user is known as the *user interface*.

The communication performed by a user interface is bidirectional.

At the simplest level, we must be able to describe our problem to the expert system, and the system must be able to respond with its recommendations.

We may want to ask the system to explain its “reasoning”, or the system may request additional information about the problem from us.

Beside these three components, there is a Working Memory - a data structure which stores information about a specific run. It holds current facts and knowledge.

Stages of expert system development

Although great strides have been made in expediting the process of developing an expert system, it often remains an extremely time consuming task.

It may be possible for one or two people to develop a small expert system in a few months; however the development of a sophisticated system may require a team of several people working together for more than a year.

An expert system typically is developed and refined over a period of several years.

We can divide the process of expert system development into five distinct stages.

In practice, it may not be possible to break down the expert system development cycle precisely.

However, an examination of these five stages may serve to provide us with some insight into the ways in which expert systems are developed.

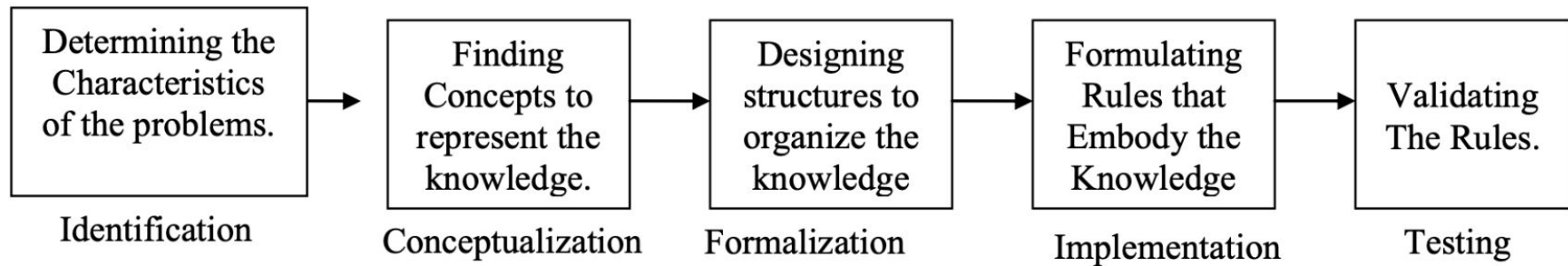


Fig: Different phases of expert system development

Identification:

Beside we can begin to develop an expert system, it is important that we describe, with as much precision as possible, the problem that the system is intended to solve.

It is not enough simply to feel that the system would be helpful in certain situation; we must determine the exact nature of the problem and state the precise goals that indicate exactly how we expect the expert system to contribute to the solution.

Conceptualization:

Once we have formally identified the problem that an expert system is to solve, the next stage involves analyzing the problem further to ensure that its specifics, as well as its generalities, are understood.

In the conceptualization stage the knowledge engineer frequently creates a diagram of the problem to depict graphically the relationships between the objects and processes in the problem domain.

It is often helpful at this stage to divide the problem into a series of sub-problems and to diagram both the relationships among the pieces of each sub-problem and the relationships among the various sub-problems.

Formalization:

In the preceding stages, no effort has been made to relate the domain problem to the artificial intelligence technology that may solve it. During the identification and the conceptualization stages, the focus is entirely on understanding the problem. Now, during the formalization stage, the problem is connected to its proposed solution, an expert system, by analyzing the relationships depicted in the conceptualization stage.

During formalization, it is important that the knowledge engineer be familiar with the following:

- The various techniques of knowledge representation and heuristic search used in expert systems.
- The expert system “tools” that can greatly expedite the development process.
And
- Other expert systems that may solve similar problems and thus may be adequate to the problem at hand.

Implementation:

During the implementation stage, the formalized concepts are programmed onto the computer that has been chosen for system development, using the predetermined techniques and tools to implement a “first pass” prototype of the expert system.

Theoretically, if the methods of the previous stage have been followed with diligence and care, the implementation of the prototype should be as much an art as it is a science, because following all rules does not guarantee that the system will work the first time it is implemented.

Many scientists actually consider the first prototype to be a “throw-away” system, useful for evaluating progress but hardly a usable expert system.

Testing:

Testing provides opportunities to identify the weakness in the structure and implementation of the system and to make the appropriate corrections.

Depending on the types of problems encountered, the testing procedure may indicate that the system was

Features of an expert system

What are the features of a good expert system? Although each expert system has its own particular characteristics, there are several features common to many systems. The following list from Rule-Based Expert Systems suggests seven criteria that are important prerequisites for the acceptance of an expert system .

1. "The program should be useful." An expert system should be developed to meet a specific need, one for which it is recognized that assistance is needed.
2. "The program should be usable." An expert system should be designed so that even a novice computer user finds it easy to use .
3. "The program should be educational when appropriate." An expert system may be used by non-experts, who should be able to increase their own expertise by using the system.

4. “The program should be able to explain its advice.” An expert system should be able to explain the “reasoning” process that led it to its conclusions, to allow us to decide whether to accept the system’s recommendations.

5. “The program should be able to respond to simple questions.” Because people with different levels of knowledge may use the system , an expert system should be able to answer questions about points that may not be clear to all users.

6. “The program should be able to learn new knowledge.” Not only should an expert system be able to respond to our questions, it also should be able to ask questions to gain additional information.

7. “The program’s knowledge should be easily modified.” It is important that we should be able to revise the knowledge base of an expert system easily to correct errors or add new information.

Knowledge Representation in expert system

1. Logic based representation

Logic-based representation uses formal logic to encode knowledge. It involves using logical expressions to define rules and relationships between concepts.

Example

First-Order Logic (FOL): Can be used to represent statements like "All humans are mortal" as $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$. This is useful in expert systems for reasoning about complex relationships.

Knowledge Representation in expert system

2. Rule based System

Rule-based systems represent knowledge using a set of "if-then" rules. These rules are derived from expert knowledge and are applied by the inference engine to reach conclusions.

Example

MYCIN: An early expert system for diagnosing bacterial infections. It used rules like "if the patient has a fever and cough, then they may have pneumonia" to recommend treatments

Knowledge Representation in expert system

3. Semantic Networks

Semantic networks represent knowledge as a graph where nodes represent concepts, and edges represent relationships between them.

Example

A semantic network might show that "cat" is a subclass of "animal," which is a subclass of "living things." This helps in reasoning about hierarchical relationships

Knowledge Representation in expert system

4. Ontology based systems

Ontology-based systems use ontologies to represent knowledge. Ontologies define a common vocabulary and structure for concepts and relationships within a domain.

Example

Biological Ontologies: In biology, ontologies are used to describe relationships between species, genes, and biological processes. This facilitates data integration and reasoning across different biological databases

Knowledge Representation in expert system

5. Frame based systems

Frame-based systems represent knowledge using frames, which are structured representations of objects or concepts. Each frame includes attributes and values related to the concept.

Example

A frame for a car might include attributes like "color," "manufacturer," and "fuel type." This is useful in expert systems for representing complex objects and their properties

Knowledge Acquisition and Learning

Knowledge acquisition and learning are fundamental processes in developing expert systems and artificial intelligence applications

Knowledge acquisition is the process of extracting, structuring, and organizing knowledge from various sources, including human experts, documents, and databases.

It is a critical step in building expert systems, as it involves capturing the expertise of domain specialists and translating it into a machine-interpretable form.

Different techniques of Knowledge Acquisition

Manual Techniques:

Interviews: Face-to-face interactions with experts to gather knowledge. Interviews can range from unstructured to highly structured.

Protocol Analysis: Analyzing the thought processes and decision-making steps of experts while they perform tasks.

Surveys and Questionnaires: Collecting information through written or online surveys.

Different techniques of Knowledge Acquisition

Semi-Automated Techniques:

Knowledge Elicitation Tools: Software tools that assist in extracting knowledge from experts, often using structured templates or questionnaires.

Different techniques of Knowledge Acquisition

Automated Techniques:

Machine Learning: Using algorithms to derive rules and patterns from data without direct human intervention.

Natural Language Processing (NLP): Automatically extracting knowledge from texts and documents.

Application of Expert System

Medical Diagnosis:

MYCIN: An early expert system for diagnosing bacterial infections and recommending antibiotics.

DXplain: A clinical decision support tool that suggests possible diseases based on symptoms.

CaDet: Assists in the early detection of cancer by analyzing patient data.

Financial Services:

Used for asset management, acting as robo-advisors, and predicting market trends.

Applications include credit scoring and fraud detection

Application of Expert System

Agriculture:

Expert systems help in crop management, forecasting crop damage, and providing decision support for farmers

Education:

Used to create personalized learning plans and support educational decision-making

Environmental Management:

Expert systems assist in monitoring and managing environmental resources, predicting environmental impacts.

Customer Service:

Automate customer support by routing requests and solving problems based on predefined rules

Forward Chaining

Forward chaining is a data-driven inference method that starts with known facts and applies rules to extract more information until a goal is reached.

It moves from the premises to the conclusion.

Example: Consider a simple medical diagnosis system where we want to determine if a patient has the flu based on certain symptoms.

Facts:

The patient has a fever (Fact 1).

The patient has a sore throat (Fact 2).

Forward Chaining

Rule: If the patient has a fever and sore throat, then they might have the flu.

In forward chaining, the system starts with the known facts (fever and sore throat) and applies the rule:

It checks if both conditions (fever and sore throat) are met.

Since both conditions are true, it concludes that the patient might have the flu.

This process continues until no new conclusions can be drawn or a specific goal is reached.

Backward Chaining

Backward chaining is a goal-driven inference method that begins with a goal or hypothesis and works backward to determine whether the available facts support that goal.

It breaks down the goal into smaller sub-goals until all necessary facts are verified or no more supporting data can be found.

Backward Chaining

Example: Prove that John is the tallest boy in his class.

Knowledge Base:

John is taller than Kim.

John is a boy.

Kim is a girl.

John and Kim study in the same class.

Everyone else in the class is shorter than Kim.

Backward Chaining

In backward chaining, we start with our goal (John is the tallest boy) and check if it can be supported by known facts:

We assume John is the tallest boy.

To validate this, we check if his height is greater than anyone else in the class.

We verify that John is indeed taller than Kim and that everyone else is shorter than Kim.

The knowledge base confirms these conditions, thus proving that John is indeed the tallest boy in class.